qwen-metal

Architecture & System Design Document

A from-scratch, single-model LLM inference engine in Swift + Metal for iPhone — Qwen ~1.5–2B, 4-bit quantized — benchmarked head-to-head against MLX Swift and llama.cpp on the same physical device.

Version 1.5 · August 25, 2026 (Phase 2 exit: naive GPU engine measured on-device) · Companion to PLAN.md, CLAUDE.md, DECISIONS.md, and the phase specs. Where this document and DECISIONS.md disagree, DECISIONS.md (the append-only log) wins.

---

1 · Purpose, Objectives, and Scope

This project builds a minimal but complete LLM inference engine for iPhone, written from scratch in Swift with custom Metal Shading Language kernels. It is deliberately the "nanoGPT of Metal inference": one model family, one quantization format, batch size one, and no abstraction layers — so that every level of the stack that mature engines hide (weight layout, kernel dispatch, quantized matrix multiply, attention over a cache) is implemented, measured, and understood by hand. The deliverable is twofold: a working on-device engine, and a rigorous benchmark writeup comparing it against the two incumbent runtimes, MLX Swift and llama.cpp, on identical hardware with a roofline analysis explaining every gap.

The performance goal is calibrated by physics rather than ambition: MLX decodes the pinned Qwen3-1.7B 4-bit checkpoint at a **measured 39.2 tok/s** warm-burst on the project's iPhone 15 Pro (Phase 0 row, PROVISIONAL), which sits essentially at the measured memory-bandwidth roofline (§5). Matching it is therefore not required for success. The committed target is absolute and now pinned (DECISIONS.md, Phase 0 exit): decode ≥ 0.75 × MLX's measured decode tok/s = **29.4 tok/s**, both sides measured in the same session at the canonical measurement window (generated tokens 128–512). Being able to account for the remaining gap is the point. As of Phase 2 exit (2026-08-25) the deliberately naive engine runs on the device and measures **6.7–8.6 tok/s** decode at bf16 weights (the 'before' row, PROVISIONAL) — 50–70% of its own bandwidth roofline; Phases 3–5 own the gap from there.

1.1 · Non-goals (scope is a feature)

Breadth is where mature engines spend most of their engineering, and it teaches little per hour invested. Each scope cut below removes multiplicative surface area while keeping exactly one instance of every core concept. These cuts are contractual: the coding agent is instructed (CLAUDE.md) to flag rather than implement anything on this list, however easy it appears.

Cut	What is kept instead	Concept still fully exercised
Architecture registry, MoE, VLM	One dense model family: Qwen 2.5/3, 1.5–2B	Full transformer stack: embeddings, RMSNorm, RoPE, GQA attention, SwiGLU
Multiple quant formats (K-quants, i-quants, …)	4-bit grouped affine only (group 64, fp16 scale+bias)	Quantized kernel design, packing, quality evaluation
Dynamic context / cache management	Fixed 4K context, KV cache preallocated at load	Cache layout, incremental decode, GQA sizing
Sampler zoo (top-p, top-k, beam…)	Greedy + temperature	Logits → token pipeline
Batching / serving	Batch size 1 always	The actual on-device workload (and why decode is bandwidth-bound)
Custom tokenizer	swift-transformers (borrowed)	— (BPE teaches nothing about GPUs)

Cut	What is kept instead	Concept still fully exercised
ANE / Core ML build target	GPU (Metal) only; Core ML appears as a benchmark column	— (ANE is a closed compiler target; no custom kernels possible)
iOS Simulator	macOS CLI for dev; physical iPhone for truth	— (simulator lacks the Metal feature set and its perf numbers are meaningless)

2 · System Context and Repository Architecture

All engine logic lives in one shared Swift package, **QwenMetalEngine**, consumed by two thin targets: a macOS command-line tool that serves as the development workhorse (tests, correctness diffs, first-pass profiling run here at desktop iteration speed), and an iOS app that exists only to host the engine on a physical iPhone for official measurements. No engine logic may live in either target — this keeps the code that is benchmarked on the phone byte-identical to the code that is tested on the Mac, which is what makes the Mac-first dev loop sound: Apple Silicon Macs expose the identical Metal API against an architecturally similar GPU, so correctness transfers even though performance numbers do not.

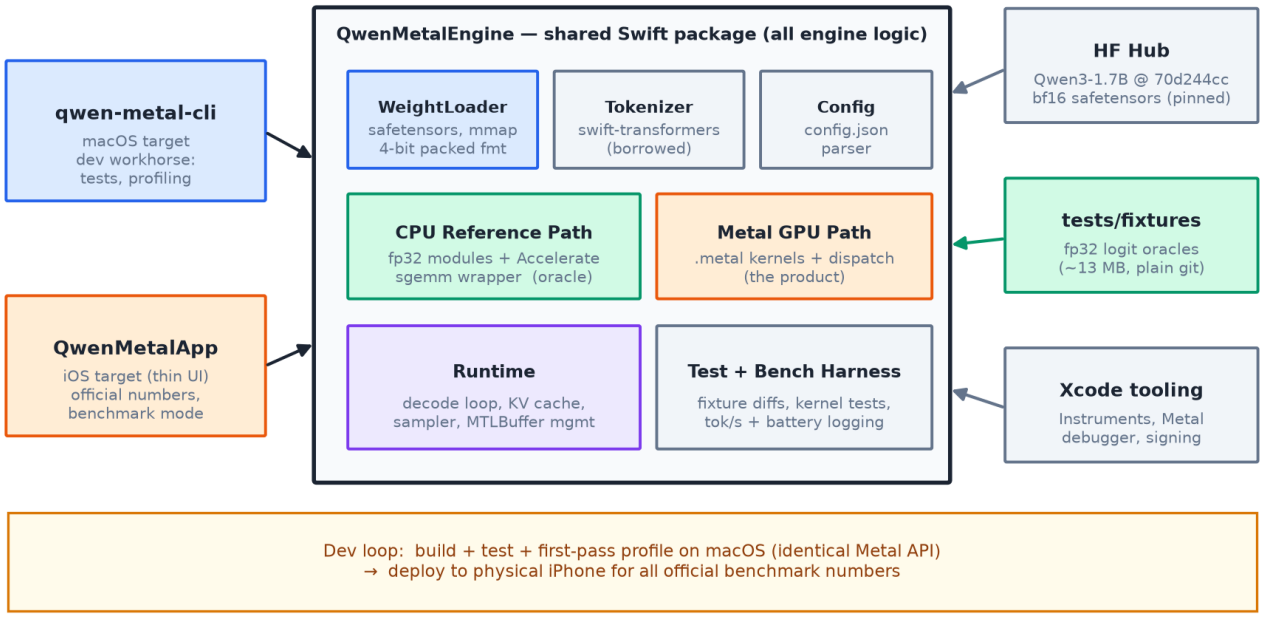

Figure 1 — System context. The shared engine package, its two consumers, external inputs (pinned checkpoint, fixtures, tooling), and the two-machine development loop.

The package divides into a weight/config layer (safetensors parsing, mmap, the offline 4-bit packing output), a borrowed tokenizer, two parallel compute paths — the CPU reference path (§6) and the Metal GPU path (the product) — and a runtime layer that owns the decode loop, the preallocated KV cache, sampling, and MTLBuffer lifetime. The test and benchmark harness sits beside the runtime and is a first-class component: the project's entire final deliverable is a credible measurement, so measurement code is engineered, not scripted.

3 · Runtime Architecture: the Decode Path

Figure 2 traces one decode step — the loop the engine lives in. A token id is embedded, passes through N transformer layers, and produces logits over the 152k-entry vocabulary, from which the next token is sampled and fed back. Two structural facts dominate the design. First, every orange block is a matrix product against quantized weights, executed by the fused dequant-matvec kernel of §5 — at batch 1 these are matrix-vector products, and together they stream essentially the entire model through the GPU once per token. Second, decode is incremental from Phase 2 onward: Keys and Values for each new position are appended to a preallocated cache, and attention reads over the cache rather than recomputing the whole sequence. This ordering is deliberate and load-bearing (Decision Register, entry 1): without the cache, per-step work grows with context and turns compute-bound within ~10 tokens, which would invalidate every bandwidth-based measurement the later phases rely on.

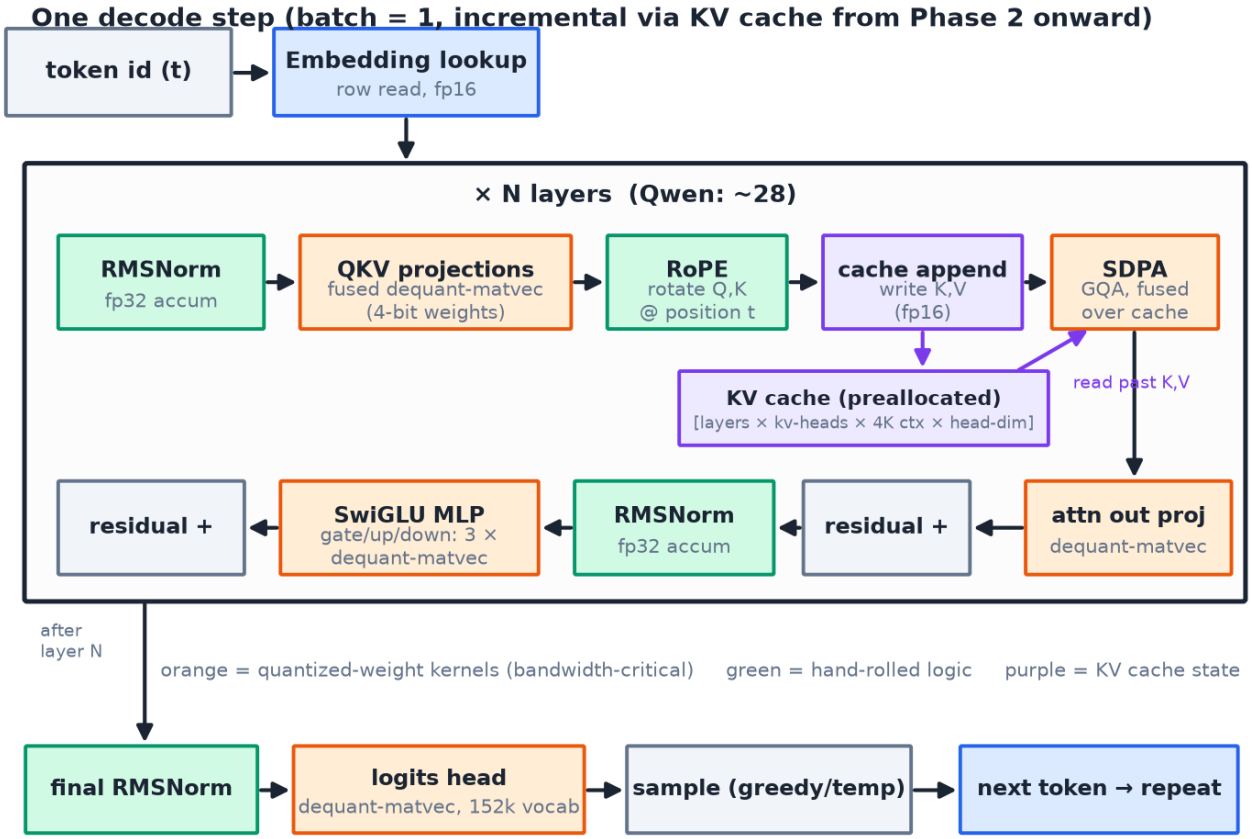

Figure 2 — One decode step. Orange = quantized-weight kernels (bandwidth-critical); green = hand-rolled numeric logic; purple = cache state. The top row (attention) runs left-to-right; the middle row (attention output through the MLP block) runs right-to-left.

Qwen's grouped-query attention (GQA) matters twice here: the attention kernel must map multiple query heads onto each shared KV head, and the cache shrinks proportionally — with far fewer KV heads than Q heads, a 4K-context fp16 cache for a ~1.5B model costs on the order of 0.1–0.2 GB rather than a multiple of it. On a device with a hard memory ceiling this is not an optimization but an enabling condition.

4 · Memory Architecture

iPhone inference runs inside two unusual memory constraints. Unified memory means CPU and GPU share one physical DRAM pool — there are no host-device copies, and an MTLBuffer in shared storage is directly visible to both sides — but also one shared bandwidth budget — measured at 43.84 GB/s sustained on the pinned iPhone 15 Pro (Phase 0 triad; A17 Pro rated 51.2). And iOS enforces a per-app ceiling well below the device's 8 GB: exceed it and the OS terminates the process ("jetsam") without ceremony. The engine therefore treats memory as a budgeted resource with a static plan, shown in Figure 3: every allocation is accounted before it is written, weights are memory-mapped so their pages are clean and file-backed (which iOS's memory accounting treats far more leniently under pressure than dirty heap pages), and nothing grows at runtime — the KV cache is carved out in full at model load. Phase 2 measured the accounting asymmetry directly (DECISIONS.md 2026-08-25): the full bf16 engine shows an on-device phys_footprint of **~536 MB under mmap** — the 3.44 GB of clean file-backed weight pages largely escape the metric — versus **~4.3 GB** with the wired-copy variant, at load times of 1.5 s vs 9.7 s. Speed between the two residency modes did not separate from device-state noise (both directions observed), so the recorded default for Phase 3+ is mmap, with the comparison re-run interleaved on the packed weights before the question is treated as closed.

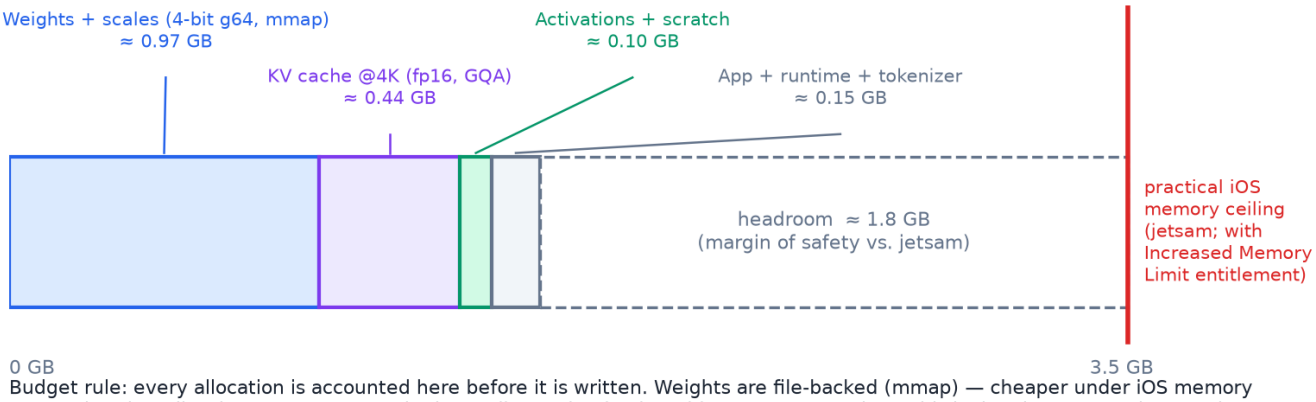

Figure 3 — Static memory budget against the practical iOS ceiling (with the Increased Memory Limit entitlement). Figures derive from the pinned Qwen3-1.7B config (DECISIONS.md PIN-1): ~0.97 GB packed 4-bit weights incl. scales, 448 MiB fp16 GQA KV at 4K (Phase 2 allocates exactly this, verified by test). The Phase 2 bf16 interim was the ~4.0 GB high-water mark and is now measured: on-device phys_footprint mmap ~536 MB vs wired-copy ~4.3 GB (DECISIONS.md 2026-08-25).

5 · Kernel Design and the Performance Model

Autoregressive decode at batch 1 is memory-bandwidth-bound: each token requires streaming all weights through the GPU while performing little arithmetic per byte. The governing equation — the decode roofline — is simply **tok/s ≈ DRAM bandwidth ÷ bytes read per token**. Bandwidth is fixed by the hardware, so the only lever the software owns is the denominator, and the two design commitments below are both denominator plays.

5.1 · The fused dequant-matvec (the money kernel)

Weights are stored in DRAM as 4-bit grouped-affine codes (groups of 64, per-group fp16 scale and bias, w ≈ scale·q + bias) and are dequantized *in registers* inside the consuming kernel: unpack two nibbles per byte with shifts and masks, apply scale and bias, multiply-accumulate into an fp32 accumulator, discard. The dequantized fp16 value exists for a few cycles inside the compute unit and never touches DRAM. The alternative — dequantizing to an fp16 scratch tensor and then multiplying — reads 0.5 B, writes 2 B, and re-reads 2 B per weight: roughly a 9× traffic penalty plus a multi-GB scratch buffer that the memory budget of §4 cannot absorb. This is why "never materialize dequantized weights" is a hard invariant rather than a guideline: fusion is not an optimization applied to the design — it is the design.

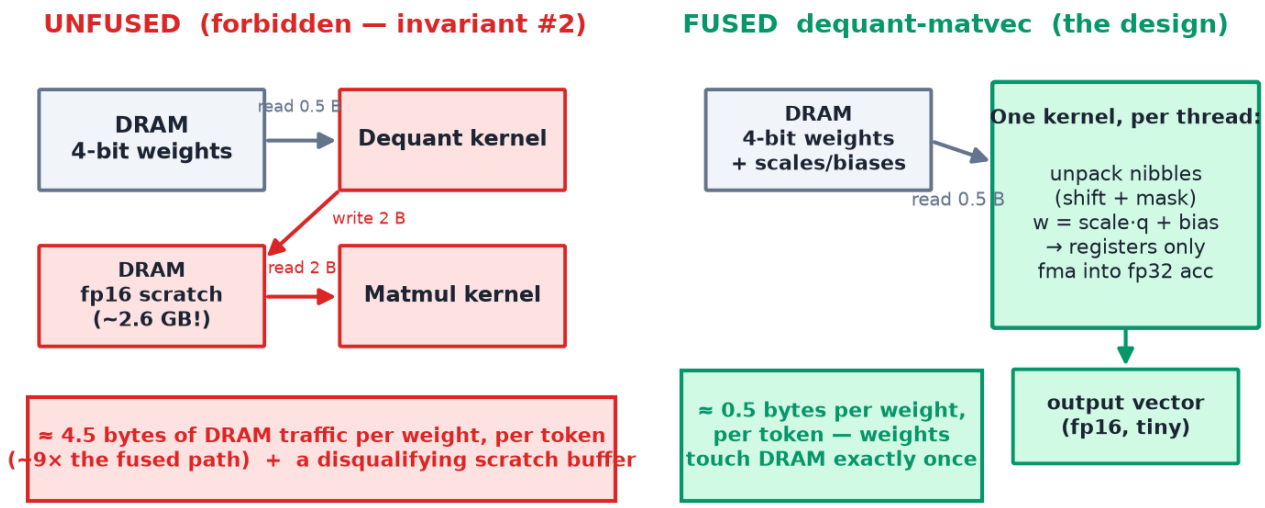

Figure 4 — DRAM traffic per weight, per token: unfused (forbidden) vs. fused. GPUs have no 4-bit arithmetic units; both paths do fp16/fp32 math — the difference is whether dequantized values live in registers or DRAM.

5.2 · Roofline: what the numbers must mean

Figure 5 plots the ceiling with Phase 0's measured numbers (DECISIONS.md 2026-08-22). A ~2B model at 4-bit reads ≈1.1–1.3 GB per token (packed weights + scales + cache), putting the ceiling near 34–44 tok/s at the measured 43.84 GB/s — and the measured engine rows land essentially on it: MLX 39.2 tok/s (~1.14 GB/token) and llama.cpp 32.4 tok/s (Q4_K_M's 5.03 bits/weight costs more bytes/token). Both operate at ~96–102% of the triad figure, which is itself a finding: decode traffic is read-dominated and a 2-read+1-write triad slightly understates read-mostly achievable bandwidth (backlog task BW-1 bounds this with a read-only variant). MLX being on the physical limit is the evidence that the target must be framed relative to its measured speed, not beating physics. The same chart shows why quantization is worth ~3× by itself (fp16 weights triple the denominator), and it gives every future benchmark number a place to stand: a result is explained when its distance from the roofline is attributed to identified causes (dispatch overhead, unfused ops, cache traffic), and suspect when it exceeds it.

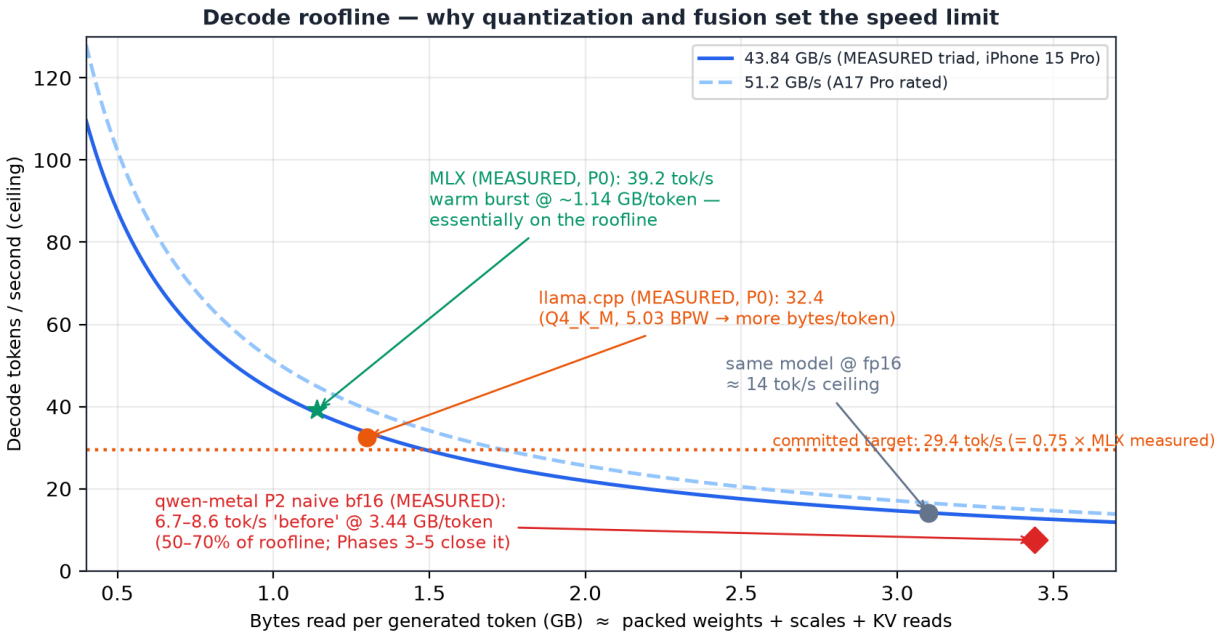

Figure 5 — Decode roofline with measured values: the 43.84 GB/s sustained triad curve, the measured MLX and llama.cpp rows, the committed 29.4 tok/s target, the fp16 counterfactual, and the Phase 2 naive bf16 'before' point (6.7–8.6 tok/s measured on-device, 2026-08-25).

Phase 2 put the first of our own points on this chart (PROVISIONAL, naive by design — no kernel has been optimized yet, per the correctness-first rule). The bf16 engine reads ~3.44 GB per token, a ~12.7 tok/s ceiling, and measured **6.7–8.6 tok/s** on-device — 50–70% of the bandwidth roofline, with a further ~1.4× run-to-run device-state variance band observed at identical settings (the Phase 3 spec must pin a repeats/interleaving protocol in response). Dispatch overhead — the Phase 4 metric — measured **1.9–2.0 ms/token at 591 dispatches/token** (~3.4 µs/dispatch), stable across every run, session, and residency mode. Projecting the observed 50–70% efficiency onto the Phase 3 packed size (~0.97 GB/token, 45.2 tok/s ceiling) lands at ~22–32 tok/s — bracketing the 29.4 target, which is the quantitative statement that Phases 4–5 remain load-bearing, not optional. Sequential prefill measured 8.2–10.7 tok/s (Phase 5's 'before' number).

Prefill obeys different physics: processing the whole prompt at once is matrix-matrix work in which each weight read is reused across all prompt positions, so it is compute-bound and rewards classical GEMM engineering — threadgroup-memory tiling and simdgroup_matrix (8×8 cooperative tile-multiply) accumulation. It is deliberately scheduled late (Phase 5) because it is a separate discipline from the bandwidth work of Phases 3–4, and a naive prefill still functions, merely slowly. The two phases are therefore always benchmarked as separate numbers.

Two secondary kernel concerns round out the design. Dispatch overhead: each kernel launch costs fixed time, so Phase 4 folds RMSNorm and RoPE into neighboring kernels and fuses scaled-dot-product attention (with the GQA head mapping) into one kernel, measured as dispatches-per-token. And ALU balance: unpacking nibbles is not free — a carelessly written fused kernel can make itself compute-bound on dequant arithmetic. The bit layout is therefore chosen so unpacking is a couple of shifts and masks with adjacent threads reading adjacent bytes (coalescing), and Instruments' limiter counters (ALU-limited vs. bandwidth-limited) are the arbiter whenever a kernel lands below its roofline prediction.

6 · Correctness Architecture: the Oracle Chain

Every performance claim in this project rests on the engine being correct, and correctness here is architectural, not aspirational: it is a directed chain of oracles in which no result is ever compared against a reference that legitimately differs from it. The chain was hardened across six planning-review findings (§9); its final shape is Figure 6.

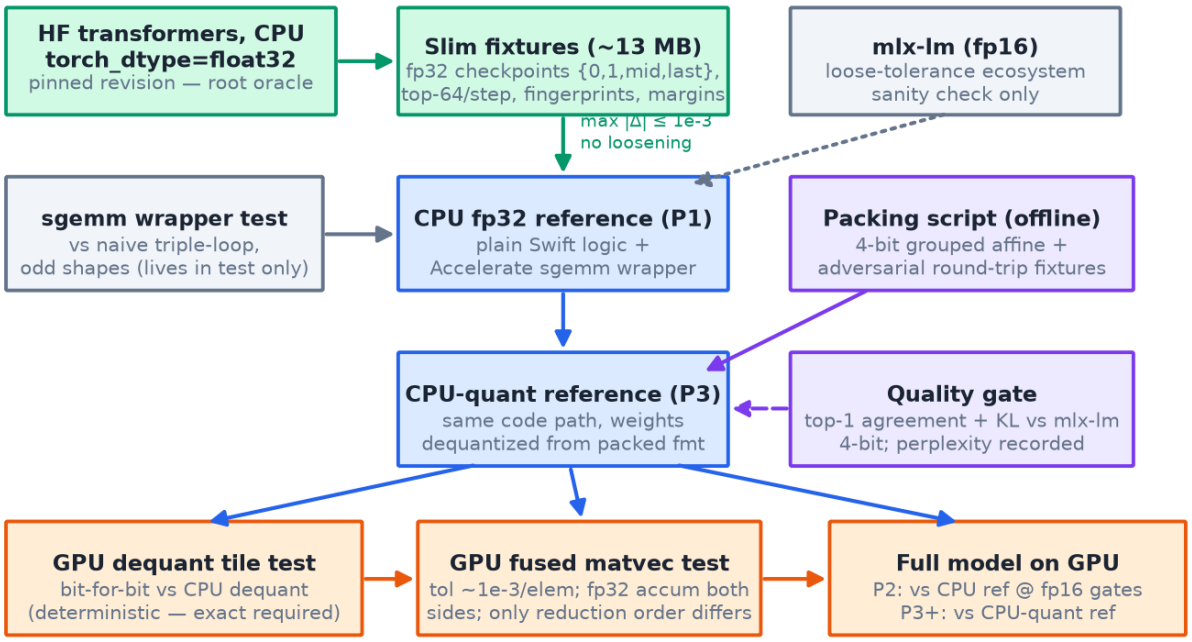

Figure 6 — The oracle chain. Green = root truth (fp32, noise-free); blue = CPU references; purple = quantization artifacts and their quality gate; orange = GPU tests. Every arrow is a test that exists.

Root oracle. HF transformers on CPU at torch_dtype=float32, pinned revision. Both our engine and the reference load identical bf16 weight bits and upcast losslessly, so the only benign divergence between them is fp32 summation order (~1e-4 on logits). The Phase 1 tolerance is therefore max |Δlogit| ≤ 1e-3 with **no loosening permitted** — failure means a bug or config mismatch, and the correct response is investigation, never a wider bound. An mlx-lm (fp16) run survives only as a loose-tolerance ecosystem sanity check.

Slim fixtures. The oracle ships as ~13 MB of plain-git fixtures rather than 150 MB of full per-step logits: full-vocab fp32 checkpoints at steps {0, 1, mid, last} per prompt (late checkpoints catch position-dependent bugs — RoPE at depth, cache boundaries), top-64 logits every step, per-step scalar fingerprints (logsumexp/mean/std) to close the tail-drift gap, and a per-step top-1/top-2 margin enabling tie-aware sequence matching: steps where the reference's margin is below epsilon are exempt from exact argmax match, which converts the flakiest failure mode of logit tests (benign near-tie flips from summation order) into a principled rule. Full oracles are regenerable locally by one documented command for forensics and are never committed. Storage is fp32 throughout — fp16 spacing (~0.016 at logit magnitude 16–32) would exceed the tolerance itself.

Quantized paths get their own oracle. From Phase 3, the GPU computes on 4-bit weights and legitimately differs from any fp reference — so it is graded against a CPU-quant reference: the same Phase 1 code path with a weight loader that dequantizes our packed format. Kernel tests are layered by where bugs live: the unpack/dequant tile test demands

bit-for-bit equality (deterministic, no reduction — this is where nibble-order, shift, group-boundary and scale-indexing bugs concentrate), while the fused matvec test allows ~1e-3/element with fp32 accumulation on both sides, since only reduction order differs once layer one has proven the inputs identical. Quantization *quality* is gated separately — top-1 agreement and mean KL versus mlx-lm at 4-bit, plus a fixed perplexity eval — against the standard of "in the same band as the reference implementation," because our layout and rounding legitimately differ from MLX's bytes. A quality-gate failure with passing kernel tests indicts the packing script, not the kernel.

The oracle must actually run. Full-recompute Phase 1 decode costs ≈10^{13} FLOPs per prompt; naive Swift matmul would put the suite at hours-to-overnight, and an unconsulted oracle protects nothing. All matmul-shaped work in the CPU reference therefore routes through a single Accelerate cblas_sgemm wrapper (~two orders of magnitude faster via the AMX units), keeping the suite in minutes. The wrapper is validated once against a naive triple-loop implementation on odd-shaped matrices (that implementation lives only inside the test), and the independent HF oracle catches any transpose/leading-dimension misuse since it shares none of our BLAS calls. All model logic — norms, RoPE, softmax, gating — remains plain hand-rolled Swift: sgemm contains no model logic, so the auditability argument survives the substitution.

Phase 1 outcome (2026-08-23) — the chain held. The CPU reference passed the full logit-match suite on the first run with every pre-committed gate unmodified: all 5 prompts × 50 teacher-forced steps, full-vocab checkpoints ≤ 1e-3, per-step fingerprints and top-64, and exact argmax at all 250 steps (no step needed the tie exemption). swift-transformers (pinned exact 1.3.3) produced token ids identical to the Python dump on all fixtures. A post-phase adversarial audit (AUDIT-1, 12 candidates → 5 confirmed) then hardened the load path: config numeric validation closed a 2^63 overflow crash and silent-NaN eps/theta acceptance; the decode stop set now unions generation_config.json's eos ids (restoring parity with HF generate(), which the teacher-forced suite structurally cannot see); and the tokenizer artifacts are sha256-pinned in the test harness. The Phase 2 fp16 gate is already pre-committed (DECISIONS.md 2026-08-23): tiered kernel/module/end-to-end tolerances derived from fp16 rounding — made clean by keeping GPU weights as the raw bf16 checkpoint bits — plus a tie-aware top-1 agreement gate over the same 250 teacher-forced steps; free-running divergence is reported, not gated.

Phase 2 outcome (2026-08-25) — the chain held again, now on the GPU. The naive Metal engine passed every pre-committed fp16 gate unmodified on the first run: the exact (==) surfaces (embedding lookup, kv-append, wired-copy vs mmap bit identity, the bf16 register upcast), Tier-K kernel diffs, Tier-M isolated modules vs the Phase 1 activation fixtures, and the Tier-E teacher-forced logit suite — full-vocab checkpoints, per-step fingerprints, top-64, and tie-aware top-1 agreement across all 250 steps (98 s on GPU vs ~32 min for the CPU suite: the KV cache at work). The free-running divergence report came back **none**: the GPU trajectory is token-identical to the CPU reference on all 5 prompts × 128 greedy steps. The decode stop set moved into the engine (ModelDirectory), closing the audit's EOS finding at its final home. The two judgment-derived Tier-E constants flagged at gate-commit time were never needed as slack — the veto window closed with every gate untouched.

7 · Benchmark and Measurement Methodology

All official numbers come from one physical iPhone; the same device, model, quant level, prompt set, and generation length are used for every engine (ours, MLX LLMEval, llama.cpp). Device model, iOS version, battery level, battery health, and starting temperature are recorded per run; runs happen in both **burst** (short generation from rest) and **sustained** (≥5 minutes continuous, via the regenerate-loop protocol: on 4K context fill, reset and regenerate from the same prompt — identically for every engine) form, because thermal throttling makes those genuinely different measurements, and cold start (weights from disk) is reported separately from warm. Cross-engine parity is pinned: greedy sampling for every engine (overriding LLMEval defaults), exact prompt strings from benchmarks/prompts/, per-engine token counts reported (GGUF and HF tokenizers tokenize the same string differently), and decode tok/s always reported at the canonical window (generated tokens 128–512) so KV-depth-dependent bytes/token cannot skew comparisons.

Metric	Definition	Instrument	Notes
Prefill tok/s	prompt tokens ÷ prefill wall time	in-app timer	compute-bound; reported separately from decode, always
Decode tok/s	generated tokens ÷ decode wall time	in-app timer	the roofline-governed headline; burst and sustained
Peak memory	high-water mark during generation	phys_footprint (Xcode memory gauge)	one metric everywhere; Instruments Allocations misses Metal resource memory — not used for headline rows
Energy (J/tok)	(run – idle baseline) joules ÷ tokens	battery-delta protocol (below)	sustained runs only; mean ± spread over ≥3 runs
Energy impact	Instruments Energy Log score	Instruments	unitless relative indicator only; never presented as energy

iOS exposes no direct power measurement — Instruments' Energy Log is a unitless relative score, and macOS's powermetrics does not exist on iPhone — so absolute energy comes from battery drain under a controlled protocol. State of charge is reported in 1% steps (~130 J on a ~13 Wh battery), so each measured run must burn ≥8–10% SoC (20–40 minutes of continuous generation) to keep quantization error acceptable. An idle baseline of identical duration, screen state, and minimum fixed brightness — airplane mode, background refresh off, unplugged — is subtracted; runs start in the same SoC band (e.g. 80%→70%, avoiding the nonlinear extremes) at the same starting temperature; rated capacity is scaled by recorded battery health; and each engine gets ≥3 repeats reported as mean ± spread. A sanity gate discards any run whose implied average power falls outside ~3–9 W. The writeup carries an explicit measurement-limitations paragraph — stated error bounds read as more credible than suspiciously precise tables. The iOS app's benchmark mode logs durations, token counts, and start/end battery level so every sustained run self-documents.

8 · Delivery Roadmap

Seven phases, each ending with something that runs; detailed specs are written just-in-time from the previous phase's logged results (a Phase 4 spec written before Phase 3's profiler data exists would encode guesses as requirements). Effort assumes ~6–10 focused hours per week; total is roughly 3–4 months to the complete benchmarked artifact, with a runnable on-device demo existing from Phase 2 onward.

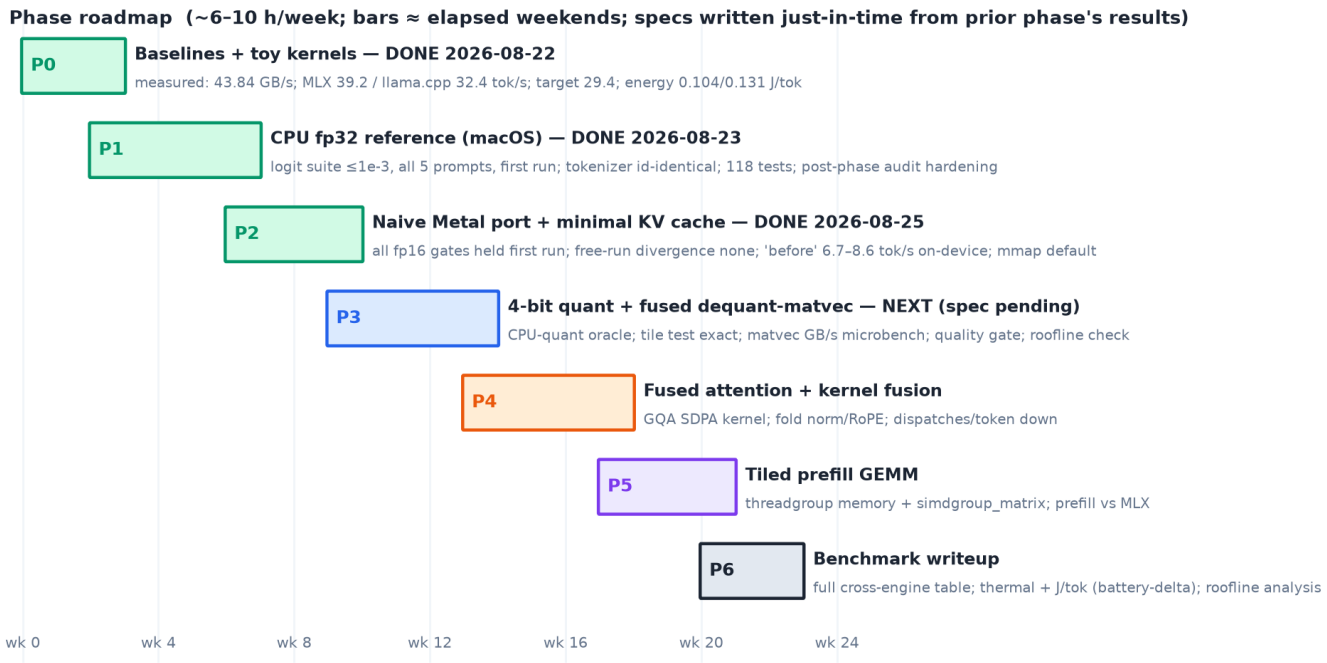

Figure 7 — Phase roadmap. Bars are elapsed calendar at part-time effort; overlap between Phase 0 and 1 is expected (baselining is largely manual device work).

Phase	Exit criterion (abridged — PLAN.md is authoritative)
0	MLX + llama.cpp baseline rows recorded on-phone; toy MSL kernels (saxpy, naive matmul) passing tests on macOS with a GPU-timing workflow
1	Full CPU fp32 forward pass; logits ≤1e-3 vs fp32 HF oracle on all fixture prompts, no loosening; mlx-lm sanity check recorded — EXITED 2026-08-23, all gates held first run
2	Incremental decode on the physical iPhone via preallocated cache + naive attention kernel; pre-committed fp16 gate vs CPU reference; 'before' row; mmap vs wired-copy comparison — EXITED 2026-08-25: all gates held first run, free-run divergence none, 'before' 6.7–8.6 tok/s, mmap default recorded
3	CPU-quant oracle exists; dequant tile test bit-exact; matvec within tolerance; quality gate passed; ~4× memory drop; matvec GB/s microbench; short-context decode near roofline with bandwidth-limited counters — NEXT (spec SPEC-P3 pending)
4	Fused GQA SDPA replaces naive attention; norm/RoPE folded into neighbors; dispatches-per-token reduced; latency-vs-context measured
5	Tiled prefill GEMM (threadgroup memory + simdgroup_matrix); prefill benchmarked separately vs MLX
6	Full cross-engine table (incl. optional Core ML column), sustained-thermal chart, J/tok with error bars, roofline analysis, honest gaps

Stretch goals, gated behind Phase 6: speculative decoding with a Qwen 0.5B draft model (turns some memory-bound decode into compute-bound verification), and upstreaming a kernel improvement to MLX or ExecuTorch's MPS backend.

9 · Design Decision Register

The plan was audited across twenty planning-review findings before any code was written (eight first-pass plus twelve outside-voice; full record in docs/reviews/2026-08-20-eng-review.md). The six most load-bearing are registered below; each produced a logged decision (DECISIONS.md, authoritative) now embedded in the architecture above. They share a theme worth keeping for the build: every finding was a case where validation machinery would have silently stopped measuring what it claimed to measure — confident wrong numbers, not visible failures. The standing review question at each phase boundary is therefore: *does every test and metric in this phase still measure what the plan says it measures?*

#	Finding	Decision	Failure prevented
1	Roofline criterion sat in Phase 3, but cache-less decode turns compute-bound past ~10 tokens of context	Minimal KV cache moved into Phase 2; Phase 3 gains a kernel-level GB/s microbenchmark; Phase 4 keeps fusion	Unmeasurable exit criterion; ALU-limited counters misread as dequant inefficiency
2	fp16-vs-fp32 diffs break when the GPU switches to 4-bit weights	CPU-quant reference (same code, packed weights); layered tests (tile bit-exact, matvec tolerance); separate quality gate vs mlx-lm 4-bit	Subtly wrong dequant kernel shipping into Phases 4–6 and poisoning every benchmark
3	Instruments' Energy Log is a unitless impact score, not joules	Battery-delta protocol for J/tok (sustained only, idle-subtracted, ≥3 repeats, sanity gate); impact score demoted to labeled relative indicator	Unreviewable energy claims in the writeup
4	Full logit oracles ≈150 MB; LFS adds toolchain and quota failure modes inside the trust-bedrock tests	Slim tiered fixtures (~13 MB plain git): fp32 checkpoints, top-64/step, fingerprints, tie-aware margins; full oracles regenerable on demand	Clone-hostile repo; silent pointer-file checkouts in CI/agent environments
5	Grading fp32 output against an fp16-computed reference leaves no gap between reference noise and real bugs	fp32 HF root oracle; tolerance tightened to 1e-3 with the loosening escape hatch deleted; fixtures stored fp32 (fp16 spacing exceeds the tolerance)	Tolerance creep until the test catches nothing; sessions burned on phantom drift
6	Naive-Swift matmul puts the oracle suite at overnight runtimes; unrun oracles protect nothing	Single Accelerate sgemm wrapper for all matmul-shaped work; model logic stays hand-rolled; wrapper validated vs triple-loop + independent HF oracle	Suite abandonment — the quiet death of the entire correctness chain

10 · Open Risks and Standing Unknowns

These are the known gaps between the plan and reality that only building will close; each has a designated first-contact point where it gets measured rather than assumed.

Risk	Exposure	First contact / mitigation
Actual device bandwidth and jetsam ceiling differ from planning estimates	Roofline targets and memory budget shift	CLOSED for bandwidth: Phase 0 measured 43.84 GB/s (2026-08-22). Memory: Phase 2 ran the ~4.0 GB bf16 high-water mark on-device without jetsam; phys_footprint asymmetry measured (mmap ~536 MB vs wired ~4.3 GB)
MLX/llama.cpp baseline numbers off published figures (different device, model rev, thermal state)	Gap-closing target mis-calibrated	CLOSED: Phase 0 measured both on-device (39.2 / 32.4 tok/s warm-burst, PROVISIONAL); target 29.4 committed from local rows
Tokenizer or chat-template mismatch vs HF reference	Phase 1 logit test fails for non-engine reasons	CLOSED: Phase 1 verified swift-transformers (pinned exact 1.3.3) id-identical to Python on all 5 fixture prompts; tokenizer artifacts sha256-pinned (TOK-1)

Risk	Exposure	First contact / mitigation
swift-transformers / mlx-swift API drift by build time	Integration friction	swift-transformers pinned exact 1.3.3; agent instructed to log and surface conflicts (CLAUDE.md), never silently work around
Thermal throttling makes sustained numbers device-state-sensitive	Noisy benchmark rows	Protocol pins starting temperature and reports burst/sustained separately; spread published, not hidden. Phase 2 measured ~1.4× run-to-run device-state variance (detached, identical settings; cold/warm not the driver) — SPEC-P3 must pin repeats/interleaving and range reporting
Scope temptation (extra quant formats, samplers, models)	Schedule and depth erosion	Non-goals are contractual; agent flags additions in DECISIONS.md instead of implementing

Document lineage: this PDF renders the state of PLAN.md, CLAUDE.md, docs/phases/phase-0-1.md, docs/phases/phase-2.md, DECISIONS.md, and benchmarks/results.md as of 2026-08-25 (Phase 2 exit) into one navigable artifact. It is a snapshot: when the build produces new measurements or decisions, DECISIONS.md is updated first and this document is regenerated from it, not edited independently.